

# CANDLESTICK VISUAL-REGIME DISCOVERY PACKAGE

*Installation, Configuration, Operation and Interpretation Manual*

## **Purpose**

To discover recurring visual regimes from Bitcoin candlestick-chart images using frozen CNN embeddings, PCA and K-Means clustering.

**Package components:** Python runner, JSON configuration, requirements file, Windows batch launchers and generated reports.

*Prepared for the MSc final-year project workflow*

# Contents

1. Purpose and scope
2. How the package fits into the project
3. Package contents
4. System and software requirements
5. Required input dataset
6. Installation
7. Configuration reference
8. Running the pipeline
9. What happens during execution
10. Output folders and files
11. Understanding cluster-selection metrics
12. Reviewing the recommended number of clusters
13. Inspecting cluster montages
14. Accepting or overriding K
15. Safe reruns and saved embeddings
16. Common errors and troubleshooting
17. Reproducibility and leakage controls
18. What to do after clustering
19. Recommended project record-keeping
20. Quick-reference checklist

## 1. Purpose and scope

This package performs the unsupervised visual-regime discovery stage of the revised MSc project. It does not forecast prices and it does not train the later future-regime classifier. Its purpose is to convert each candlestick chart into a numerical CNN embedding, reduce the feature space with PCA, group visually similar charts with K-Means, and produce interpretable reports and image montages.

### Core principle

The clusters are created without target labels. They are visual groups derived from the candlestick images. A later supervised dataset will use the cluster of a genuinely future window as the prediction target.

- Input: candlestick images only, normally exported from images/01\_kline.jpg.
- Feature extractor: ImageNet-pretrained ResNet-18 with its classification layer removed.
- Dimensionality reduction: PCA fitted on training embeddings only.
- Clustering: K-Means tested over several candidate values of K.
- Outputs: saved embeddings, PCA and K-Means models, cluster assignments, metrics, plots, representatives and montages.

## 2. How the package fits into the project

Existing OHLC CSV and chart-generation pipeline

|

v

Candlestick-only dataset preparation

|

v

THIS PACKAGE: CNN embeddings -> PCA -> K-Means regimes

|

v

Future-window label construction

|

v

Supervised CNN future-regime classifier

|

v

Comparison with majority, persistence and transition baselines

This ordering is important. Do not train the future classifier until the clustering output has been inspected and a defensible number of clusters has been accepted.

## 3. Package contents

| File                                 | Purpose                                                                                                        |
|--------------------------------------|----------------------------------------------------------------------------------------------------------------|
| run_unsupervised_regime_discovery.py | Main program. Performs embedding extraction, PCA, candidate-K evaluation, final fitting and report generation. |
| config.json                          | All user-editable paths and experiment settings.                                                               |
| requirements.txt                     | Python package requirements.                                                                                   |
| install_requirements.bat             | Windows convenience launcher for dependency installation.                                                      |

|                  |                                                      |
|------------------|------------------------------------------------------|
| run_pipeline.bat | Windows convenience launcher for the main pipeline.  |
| README.md        | Compact package summary and quick-start information. |

## 4. System and software requirements

### 4.1 Hardware

- 64-bit Windows computer is recommended because your current paths and batch files are Windows-oriented.
- At least 8 GB of RAM is recommended; 16 GB is preferable for tens of thousands of images.
- A GPU is optional. The package is CPU-safe, but embedding extraction is faster on CUDA.
- Allow several gigabytes of free disk space for copied images, embeddings, models and montage outputs.

### 4.2 Software

- Python 3.10 or 3.11 is recommended.
- pip must be available in the same Python environment used to run the package.
- Internet access is normally required on the first run to download pretrained ResNet-18 weights. Later runs can use the cached weights.
- Packages required: NumPy, pandas, Pillow, Matplotlib, scikit-learn, joblib, PyTorch and torchvision.

```
numpy>=1.24
pandas>=2.0
Pillow>=9.5
matplotlib>=3.7
scikit-learn>=1.3
joblib>=1.3
torch>=2.0
torchvision>=0.15
```

#### Environment warning

Install and run everything from the same Python environment. A common Windows problem is installing packages with one Python executable and running the script with another.

## 5. Required input dataset

The package expects the output created by `prepare_unsupervised_candlestick_dataset.py`. The dataset directory must contain the following structure:

```

btc_1h_unsupervised_candlestick_dataset/
|-- images/
|   |-- sample_0_29.jpg
|   |-- sample_1_30.jpg
|   `-- ...
`-- manifests/
    |-- train_samples.csv
    |-- validation_samples.csv
    |-- all_samples.csv
    `-- dataset_summary.json

```

The training and validation manifests must contain at least these columns:

- sample\_name
- start\_index
- stop\_index
- exported\_image

### Leakage control

The package fits PCA and K-Means only on train\_samples.csv. Validation images are transformed and assigned to the fitted training clusters. Do not merge the two manifests before running the package.

## 6. Installation

### 6.1 Extract the package

1. Download candlestick\_regime\_discovery\_package.zip.
2. Right-click the ZIP file and select Extract All.
3. Place the extracted folder in a stable location, for example C:  
\Users\m0zim\OneDrive\Desktop\final\_year\_project\_source\_codes\candlestick\_regime\_discovery\_package.
4. Avoid moving the folder while a run is in progress.

### 6.2 Install dependencies with the batch file

5. Open the extracted folder.
6. Double-click install\_requirements.bat.
7. Wait for pip to finish.
8. Read any error message before closing the window.

### 6.3 Alternative Command Prompt installation

```

cd "C:\path o\candlestick_regime_discovery_package"
python -m pip install -r requirements.txt

```

## 6.4 Verify the installation

```
python -c "import torch, torchvision, sklearn, pandas, PIL; print('Installation OK')"
```

A successful check prints Installation OK without an exception.

## 7. Configuration reference

Open config.json in a plain-text editor. JSON syntax is strict: use double quotation marks, separate fields with commas, and escape Windows backslashes as double backslashes.

```
{
  "dataset_dir": "C:
\\Users\\m0zim\\OneDrive\\Desktop\\final_year_project_2_datasets\\btc_1h_unsupervised_candlestick_dataset",
  "output_dir": "C:\\Users\\m0zim\\OneDrive\\Desktop\\final_year_project_2_outputs\\candlestick_regime_discovery",
  "candidate_k": [4, 6, 8, 10, 12],
  "selected_k": null
}
```

| Setting                | Recommended initial value        | Meaning                                                                          |
|------------------------|----------------------------------|----------------------------------------------------------------------------------|
| dataset_dir            | Path to prepared dataset         | Root containing images and manifests.                                            |
| output_dir             | New or reusable output folder    | Where embeddings, models and reports are saved.                                  |
| train_manifest         | manifests/train_samples.csv      | Training manifest used for fitting.                                              |
| validation_manifest    | manifests/validation_samples.csv | Out-of-sample validation manifest.                                               |
| batch_size             | 16                               | Number of images processed together. Reduce to 8 or 4 if memory is insufficient. |
| num_workers            | 0                                | Safest value for Windows and OneDrive paths.                                     |
| cpu_threads            | 4                                | Maximum PyTorch CPU threads.                                                     |
| device                 | auto                             | Uses CUDA when available, otherwise CPU.                                         |
| embedding_model        | resnet18                         | Currently the only supported encoder in this package.                            |
| use_pretrained_weights | true                             | Uses ImageNet-pretrained visual features. Keep true.                             |
| pca_components         | 50                               | Reduced feature dimensions. The code limits this safely                          |

|                            |               |                                                                  |
|----------------------------|---------------|------------------------------------------------------------------|
|                            |               | when necessary.                                                  |
| candidate_k                | [4,6,8,10,12] | Cluster counts to evaluate.                                      |
| selected_k                 | null          | Uses automatic recommendation. Set an integer later to override. |
| kmeans_n_init              | 20            | Independent K-Means initializations; improves robustness.        |
| stability_repeats          | 5             | Repeated fits for adjusted-Rand stability analysis.              |
| stability_subsample        | 4000          | Maximum training samples used in stability checks.               |
| silhouette_subsample       | 5000          | Maximum samples used for silhouette calculation.                 |
| nearest_images_per_cluster | 16            | Nearest-to-centroid images copied and montaged.                  |
| random_images_per_cluster  | 16            | Random members copied and montaged.                              |
| overwrite_output           | false         | Prevents automatic deletion of existing outputs.                 |
| reuse_saved_embeddings     | true          | Reuses compatible saved embeddings on later runs.                |
| pause_on_exit              | true          | Keeps the console open after completion or failure.              |

## 8. Running the pipeline

### 8.1 Before starting

- Confirm that dataset\_dir exists.
- Confirm that train\_samples.csv and validation\_samples.csv exist.
- Confirm that exported\_image paths in the manifests still point to real files.
- Choose an output\_dir that is not being used by another running process.
- Close image viewers that may lock files inside the output folder.

### 8.2 Run with the batch file

9. Double-click run\_pipeline.bat.
10. Leave the command window open.
11. Watch the progress counters during embedding extraction.

12. Wait for REGIME DISCOVERY COMPLETE.

### 8.3 Run from Command Prompt

```
cd "C:\path o\candlestick_regime_discovery_package"  
python run_unsupervised_regime_discovery.py
```

## 9. What happens during execution

### 9.1 Manifest validation

The program loads the chronological manifests, checks required columns and sorts samples by start and stop indices.

### 9.2 Pretrained encoder loading

ResNet-18 is loaded, its final fully connected classifier is replaced with an identity layer, gradients are disabled and the model is placed in evaluation mode.

### 9.3 Embedding extraction

Each candlestick image is resized and normalized using the pretrained model transformation. The resulting 512-dimensional vector is saved in chronological order.

### 9.4 PCA fitting

PCA is fitted only on training embeddings. The same fitted PCA transforms validation embeddings.

### 9.5 Candidate-K evaluation

K-Means is fitted for each candidate K and multiple quality/stability measures are calculated.

### 9.6 K recommendation

A combined normalized score ranks the candidate values using silhouette, Davies-Bouldin, Calinski-Harabasz, stability and cluster-balance information.

### 9.7 Final clustering

The selected K-Means model is fitted to all training PCA features. Validation samples are assigned with `model.predict()`.

### 9.8 Visual inspection output

The program identifies the image nearest each centroid, copies nearest and random members, and creates montage images.

## 10. Output folders and files

```
candlestick_regime_discovery/
|-- embeddings/
|-- models/
|-- reports/
|-- plots/
`-- clusters/
```

### 10.1 embeddings

- train\_embeddings.npy and validation\_embeddings.npy: original 512-dimensional CNN features.
- train\_pca\_features.npy and validation\_pca\_features.npy: PCA-reduced features.
- train\_metadata.csv and validation\_metadata.csv: manifests saved alongside embeddings to verify compatibility.

### 10.2 models

- pca.joblib: fitted training PCA transformer.
- kmeans.joblib: fitted final K-Means model.

### 10.3 reports

- candidate\_k\_metrics.csv/json: raw quality metrics for every K.
- candidate\_k\_recommendation\_scores.csv: normalized combined ranking.
- selected\_k.json: recommended K, selected K and selection source.
- train\_cluster\_labels.csv: training sample cluster IDs and centroid distances.
- validation\_cluster\_labels.csv: validation assignments using the fitted training model.
- cluster\_summary.csv: training/validation sizes, proportions and representative information.
- run\_summary.json: concise experiment configuration and key outputs.

### 10.4 plots

- k\_selection\_silhouette.png
- k\_selection\_davies\_bouldin.png
- k\_selection\_calinski\_harabasz.png
- k\_selection\_inertia.png
- k\_selection\_stability.png
- k\_selection\_recommendation.png

### 10.5 clusters

Each cluster folder contains a representative image, CSV membership lists, copied nearest and random images, and two montage files. These are the main materials for qualitative interpretation.

## 11. Understanding cluster-selection metrics

| Metric | Direction | What it measures | Important caution |
|--------|-----------|------------------|-------------------|
|--------|-----------|------------------|-------------------|

|                            |                         |                                                                                 |                                                                            |
|----------------------------|-------------------------|---------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Silhouette score           | Higher                  | Cohesion within clusters compared with separation from neighbouring clusters.   | Can favour simpler structures and is calculated on a subsample for speed.  |
| Davies-Bouldin index       | Lower                   | Average similarity between each cluster and its most similar alternative.       | Do not interpret independently of cluster sizes and visual quality.        |
| Calinski-Harabasz score    | Higher                  | Between-cluster dispersion relative to within-cluster dispersion.               | Often increases with some larger K values.                                 |
| Inertia                    | Lower; inspect elbow    | Total squared distance from samples to assigned centroids.                      | Always decreases as K increases, so the absolute minimum is not the goal.  |
| Stability ARI              | Higher                  | Agreement between repeated K-Means fits after accounting for label permutation. | High stability can still describe uninteresting clusters.                  |
| Minimum cluster proportion | Avoid tiny values       | Size of the smallest cluster relative to training data.                         | Very small classes are difficult for the later classifier.                 |
| Normalized cluster entropy | Higher is more balanced | How evenly samples are distributed across clusters.                             | Perfect balance is not required if genuine regimes have unequal frequency. |

## 12. Reviewing the recommended number of clusters

13. Open reports/candidate\_k\_metrics.csv.
14. Open reports/candidate\_k\_recommendation\_scores.csv.
15. Inspect all plots in the plots folder.
16. Compare the smallest and largest cluster proportions.
17. Inspect montages for the automatically selected K.
18. Check whether each cluster has a coherent recurring visual structure.
19. Prefer a smaller K when two choices have similar quality but the larger K creates tiny or redundant groups.

### **Do not treat the automatic K as final**

The recommendation is an engineering aid, not a scientific proof. The accepted K should

combine quantitative metrics, stability, cluster size and qualitative coherence.

## 13. Inspecting cluster montages

For every cluster, inspect both `montage_nearest.png` and `montage_random.png`.

| Montage         | Purpose                                                                                    |
|-----------------|--------------------------------------------------------------------------------------------|
| Nearest montage | Shows the most central examples and therefore the cleanest expression of the cluster.      |
| Random montage  | Shows the broader diversity and reveals whether the cluster is coherent beyond its centre. |

- Look for recurring trend direction, candle-body pattern, wick structure, consolidation, reversals and red/green balance.
- Check whether clustering is dominated by blank margins, rendering artefacts or simple colour counts.
- Compare clusters for genuine visual distinction rather than merely slight variations.
- Do not assign economic labels until later numerical statistics support them.

## 14. Accepting or overriding K

The first run normally uses:

```
"selected_k": null
```

After reviewing the metrics and montages, you may force a candidate value that was already evaluated:

```
"selected_k": 6
```

Rerun the package. The `selected_k.json` report will state whether K came from the configuration or the automatic recommendation.

## 15. Safe reruns and saved embeddings

### 15.1 Reusing embeddings

With `reuse_saved_embeddings` set to true, the program compares the saved metadata with the current manifest. Compatible embeddings are reused, avoiding the slow CNN pass.

### 15.2 When embeddings must be regenerated

- Images or image paths changed.
- The manifest order or sample set changed.
- A different embedding model or pretrained-weight setting is used.
- The saved arrays are missing or corrupted.

### 15.3 Output overwrite behaviour

- `overwrite_output=false` preserves an existing output directory.
- `overwrite_output=true` deletes the complete output directory before starting.
- Never enable overwrite while the output folder contains results that have not been backed up.

## 16. Common errors and troubleshooting

| Problem                            | Action                                                                                                                                                              |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Dataset directory does not exist   | Correct <code>dataset_dir</code> in <code>config.json</code> . Ensure the extracted dataset has not been moved.                                                     |
| Manifest does not exist            | Confirm <code>train_manifest</code> and <code>validation_manifest</code> are relative to <code>dataset_dir</code> and use forward slashes or escaped backslashes.   |
| Image does not exist               | Open the CSV and inspect <code>exported_image</code> . Rerun dataset preparation if the images were moved after the manifest was created.                           |
| <code>JSONDecodeError</code>       | Check commas, double quotes and escaped Windows backslashes in <code>config.json</code> .                                                                           |
| No module named ...                | Run <code>python -m pip install -r requirements.txt</code> using the same Python executable.                                                                        |
| CUDA requested but unavailable     | Set device to <code>auto</code> or <code>cpu</code> .                                                                                                               |
| Out of memory                      | Reduce <code>batch_size</code> from 16 to 8, 4 or 2. Keep <code>num_workers</code> at 0 on Windows.                                                                 |
| Pretrained weights cannot download | Check internet access. Run once on a connected network so <code>torchvision</code> can cache the weights.                                                           |
| Output directory conflict          | Use a new <code>output_dir</code> , set <code>reuse_saved_embeddings=true</code> , or deliberately set <code>overwrite_output=true</code> after backing up results. |
| OneDrive file lock                 | Close image viewers and editors, pause synchronization temporarily if permitted, or place the active output folder outside OneDrive.                                |
| Silhouette calculation is slow     | Reduce <code>silhouette_subsample</code> while keeping it large enough to represent the training set.                                                               |
| Very small clusters                | Use fewer clusters or inspect whether K-Means is separating artefacts rather than regimes.                                                                          |
| All clusters look similar          | The embeddings may not separate the domain well. Consider grayscale comparison, a                                                                                   |

|                                                   |                                                                                                                |
|---------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
|                                                   | different encoder or numerical regime features as a later experiment.                                          |
| Validation cluster distribution is very different | This may reflect temporal regime change. Record it rather than silently refitting clusters on validation data. |

## 17. Reproducibility and leakage controls

- Keep random\_seed unchanged for the main reported run.
- Record the exact package version, config.json and Python package versions.
- Fit PCA and K-Means on training images only.
- Do not choose K using final test data.
- Do not refit cluster centroids after viewing test performance.
- Keep the final test period untouched until clustering and future-classifier decisions are complete.
- Retain candidate-K reports even when a different K is selected manually.
- Use chronological manifests and preserve sample start/stop indices.

```
python --version
python -m pip freeze > environment_packages.txt
```

## 18. What to do after clustering

20. Inspect all cluster reports and montages.
21. Accept a defensible value of K and archive the final PCA and K-Means models.
22. Create a future-regime dataset by pairing each current window with the cluster of a non-overlapping future window.
23. Calculate class frequencies and the majority baseline.
24. Calculate the persistence baseline: predict that the future regime equals the current regime.
25. Calculate the transition-matrix baseline: predict the most frequent future cluster conditional on the current cluster.
26. Train the supervised candlestick-image classifier only after these baselines are known.
27. Evaluate with accuracy, balanced accuracy, macro F1, per-class precision/recall and a confusion matrix.

### Recommended temporal target

For a 30-candle input, use the cluster of the next non-overlapping 30-candle window as the primary target. Avoid treating sample\_1\_30 as a meaningful future target for sample\_0\_29 because the two images share 29 candles.

## 19. Recommended project record-keeping

- Copy config.json into each dated experiment folder.
- Preserve run\_summary.json and selected\_k.json.
- Keep all candidate-K plots and CSV reports, not only the selected result.
- Record whether the selected K was automatic or manually overridden.
- Keep representative and random montages for every cluster.
- Write a short interpretation note for each cluster without overclaiming economic meaning.
- Keep a table of Python, PyTorch, torchvision and scikit-learn versions.
- Back up final pca.joblib and kmeans.joblib before proceeding to supervised labels.

## 20. Quick-reference checklist

- ☐ Prepared candlestick-only dataset exists.
- ☐ Training and validation manifests exist and point to valid images.
- ☐ Package is extracted.
- ☐ Dependencies are installed in the active Python environment.
- ☐ dataset\_dir and output\_dir are correct.
- ☐ use\_pretrained\_weights is true.
- ☐ selected\_k is null for the exploratory first run.
- ☐ Pipeline completes without an exception.
- ☐ Candidate-K CSVs and plots are reviewed.
- ☐ Nearest and random montages are inspected for every cluster.
- ☐ Final K is accepted or deliberately overridden.
- ☐ PCA and K-Means models are backed up.
- ☐ No final-test images were used to fit or choose the clusters.

## Appendix A. Essential commands

```
# Install packages
python -m pip install -r requirements.txt

# Verify imports
python -c "import torch, torchvision, sklearn, pandas, PIL; print('Installation OK')"
```

```
# Run the package
python run_unsupervised_regime_discovery.py
```

```
# Record environment
python --version
python -m pip freeze > environment_packages.txt
```

## Appendix B. Minimum configuration example

```
{
  "dataset_dir": "C:
\Users\m0zim\OneDrive\Desktop\final_year_project_2_datasets\btc_1h_unsupervised_candlestick_dataset",
  "output_dir": "C:\Users\m0zim\OneDrive\Desktop\final_year_project_2_outputs\candlestick_regime_discovery",
  "train_manifest": "manifests/train_samples.csv",
  "validation_manifest": "manifests/validation_samples.csv",
  "image_column": "exported_image",
  "batch_size": 16,
  "num_workers": 0,
  "cpu_threads": 4,
  "device": "auto",
  "embedding_model": "resnet18",
  "use_pretrained_weights": true,
  "pca_components": 50,
  "candidate_k": [4, 6, 8, 10, 12],
  "selected_k": null,
  "reuse_saved_embeddings": true,
  "overwrite_output": false,
  "pause_on_exit": true
}
```

### Manual scope

This manual documents the supplied package as built. It does not claim that the discovered clusters are economically meaningful until visual inspection and later numerical validation support that conclusion.